Project 5 (in C++): Given a file contains integers, the task is to write a program to sort the integers in the file in ascending order and then outputs the sorted integers to another file. The method for sorting is the selection-sort algorithm and an 1D vector array will be used to manipulate in sorting process.

What you have to do:
1. Implement your program according to the specs given below.
2. Compile and debug your program until your program passes compilation.
3. You will be given two data files: data1 and data2; data1 contains a few integers; data2 is contains a larger set.
3. Run your program using data1 and eyeball the program's output for correctness (see if integers are sorted.)
4. When your program produces correct output from data1, then run your program using data2.

*** Include in your hard copy as follows:
- Cover page
- Source code
- inFile for data1 // with caption *** below is the input data1 ***
- outFile for data1 // with caption *** below is the sorted input data1 ***
- logFile for data1 // with caption *** below is the program logFile for data1 *** //Limit to 3 pages if more.
- inFile for data2 // with caption *** below is the input data2 ***
- outFile for data2 // with caption *** below is the sorted input data2 ***
- logFile for data2 // with caption *** below is the program logFile for data2 *** //Limit to 3 pages if more.

****
Language: C++
****
Project Name: Selection sort for integers using 1D vector
Project points: 10 pts
Due Date: Soft copy (*.zip) and hard copies (*.docx):
(10/10 pts): on time, 10/17/2026, Saturday before midnight (11:59pm)
(-10/10 pts): non-submission, 10/17/2026, Saturday after midnight

*** Name your soft copy and hard copy files using the naming convention given in Project Submission Requirements.

*** All on-line submission MUST include Soft copy (*.zip) and hard copy (*.docx) in the same email attachments with correct email subject as below; otherwise, your submission will be rejected.

Email subject: (313) first name last name <Project 5: Selection sort for integers using 1D vector (C++)>

*** Include in the email body:
- Your answers to the 5 questions (in the Project Submission Requirements.) (-1 if not include.)
- Screen recoding link. (-2 if not submit screen recording).

*** Place your screen recording in your project submission email body below the 5 questions.

****
I. Inputs:
****
a) inFile (argv [1]): a text file contains integers, not in any particular format. For example:

```
23  15  8
10  9
999 20
76 11
```

****
II. Output
****

a) outFile (argv [2]): i) Print the input data store in array before sorting; for example, the outFile should have:

** The content of array before sorting **

Index	data
0	23
1	15
2	8
3	10
4	9

ii) Print the data in array after sorting; for example, the outFile should have:

** The content of array after sorting**

Index	data
0	8
1	9
2	10
3	15
4	23

b) logFile (argv [3]: As the specs dictates.

****
III. Data structures:
****

- A sort class:

vector<int> dataAry;

method:

- constructor (…) // do as you wish.
- loadData (…) // read data from inFile and push data into dataAry. Algorithm is given below.
- selectionSort (…) // Algorithm is given below.
- printDataAry (dataAry, fileOut) // Re-use code in previous project; use dataAry.size() as count in the method.

****
IV. main (…) #include <vector>
using namespace std;
****

Step 0: check if argc count is correct and all files can be opened.
inFile ← open input file from args [0]
outFile ← open from args [1]
logFile ← open from args [2]

step 1: loadData (inFile, dataAry, logFile)

step 6: outFile ← "** The content of dataAry before sorting **" //one text line
outFile ← "Index data" // one text line
outFile ← "==========" // one text line
printDataAry (dataAry, outFile)

step 7: int count ← dataAry.size()
selectionSort (dataAry, count, logFile)

step 8: outFile ← "** The content of array after sorting **" //one text line
outFile ← "Index data" // one text line
outFile ← "==========" // one text line
printDataAry (dataAry, outFile)

step 9: close all files

```
************************************
V.  loadData (inFile, dataAry, logFile)
************************************
step 0: logFile ← "*** Entering loadData () ***"
step 1:  int data ← inFile
step 2:  dataAry ← push data onto dataAry // i.e., dataAry.push_back (data)
step 3: logFile ← "** The content of dataAryafter push another data  **" //one text line
          logFile ← "Index           data" // one text line
          logFile ← "=============" // one text line
          printDataAry (dataAry, logFile)
step 4: repeat step 1 and step 3 while inFile is not empty
         // use while (inFile >> data) for step 1 to 4
step 5: logFile ← "*** Leaving loadData () ***"

************************************
VI.  selectionSort (dataAry, count, logFile)
************************************
step 0: logFile ← "*** Entering selectionSort () ***"
step 1: i ← 0
step 2: minVal ← dataAry[i]
         minIndex ← i
         j ← i+1
step 3: if dataAry[j] < minVal
                minIndex ← j
                minVal ← dataAry[j]
step 4: j++
step 5: repeat step 3 to step 4 while j < count

step 6: if minIndex != i
                tmp ← dataAry [i]
                dataAry [i] ← dataAry [minIndex]
                dataAry [minIndex] ← tmp


step 7: logFile ← "** content of array after" i "iteration of selection sort \n" // write i.
          logFile ← "Index           data" // one text line
          logFile ← "=============" // one text line
step 8:  printDataAry (dataAry, logFile)
step 9: i++
step 10: repeat step 2 to step 9 while i < count-1

step 11: logFile ← "***Lleaving selectionSort ()"
```